

ARRAY

DINÂMICO (ArrayList)

Resolução de Problemas
Estruturados em Computação



/PARA RELEMBRAR

Array

É uma estrutura estática, ou seja, seu tamanho é definido no início do programa e não pode ser alterado.

Lista

É uma estrutura dinâmica, ou seja, é possível modificar o seu tamanho durante o tempo de execução.

/ArrayList

Se o array é uma estrutura estática, e a lista é uma estrutura dinâmica, como exatamente funciona o ArrayList? Seria ele a junção dos dois mundos?

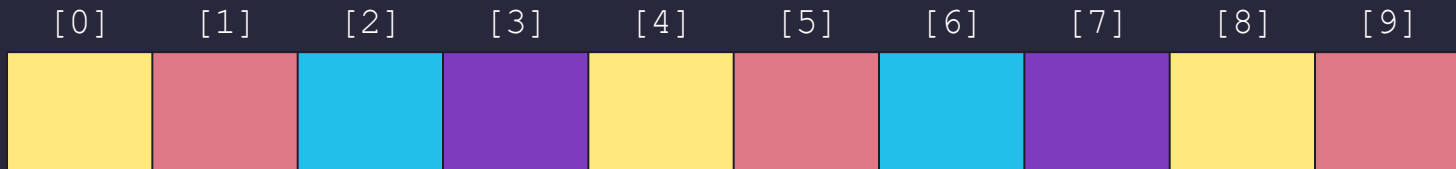
O ArrayList é uma classe da biblioteca *util* do Java, que utiliza um array estático como base para criar uma lista dinâmica.

Vamos entender um pouco como o **ArrayList** funciona...

/CRIANDO O OBJETO

```
ArrayList myList = new ArrayList();
```

Toda vez que alocamos o objeto **ArrayList** na memória, o que acontece por “trás dos panos” é a criação de um array estático com tamanho 10*.

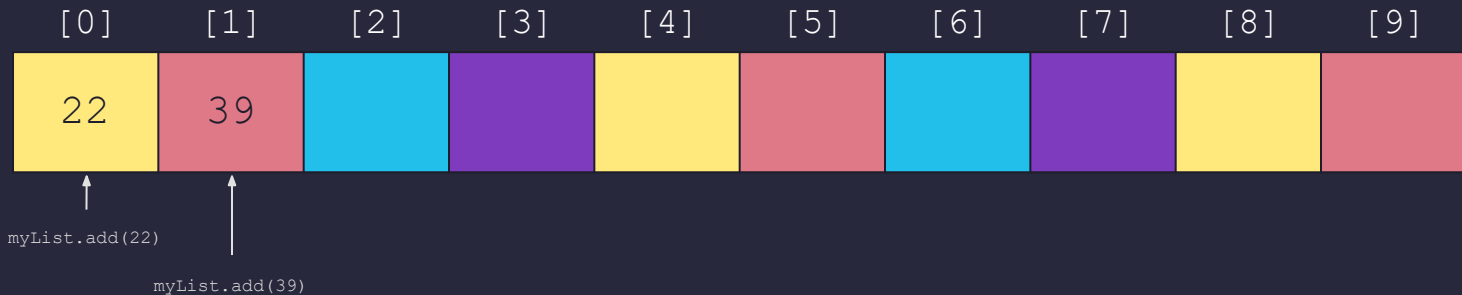


*O tamanho pode variar de acordo com a versão do Java.

/INSERINDO ELEMENTOS

```
myList.add(e)
```

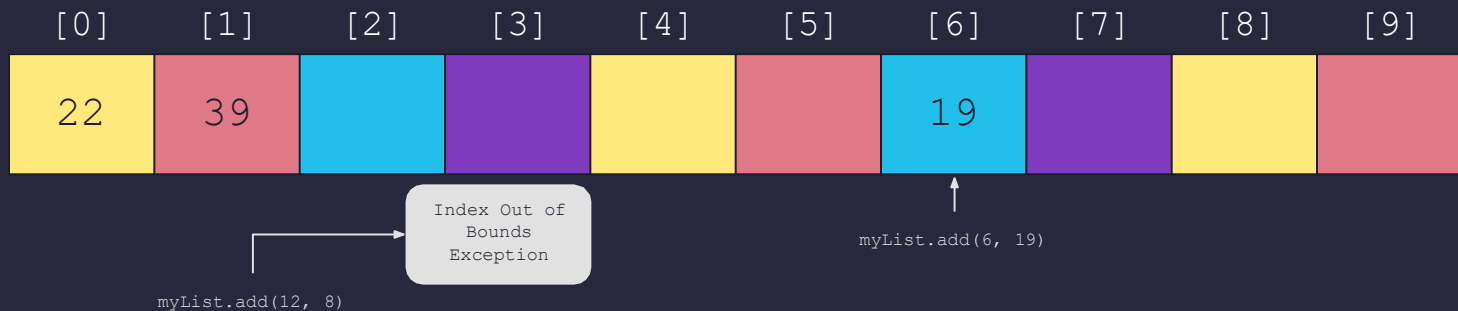
Existem alguns métodos para adicionar elementos. Um deles é o método *add*. No caso do exemplo, ele adiciona o elemento na primeira posição disponível, em ordem crescente (esquerda para direita)



/INSERINDO ELEMENTOS

```
myList.add(idx, e)
```

Neste outro exemplo, podemos definir em qual índice queremos inserir o elemento. Se o índice for válido o elemento é inserido na posição, caso contrário, o programa dispara um erro (*IndexOutOfBoundsException*).



/INSERINDO ELEMENTOS

```
myList.add(50)
```

Agora, tendo em vista que o ArrayList criou um array estático de tamanho 10 quando alocamos o objeto na memória, o que acontece quando eu tento adicionar um novo elemento com o array cheio?

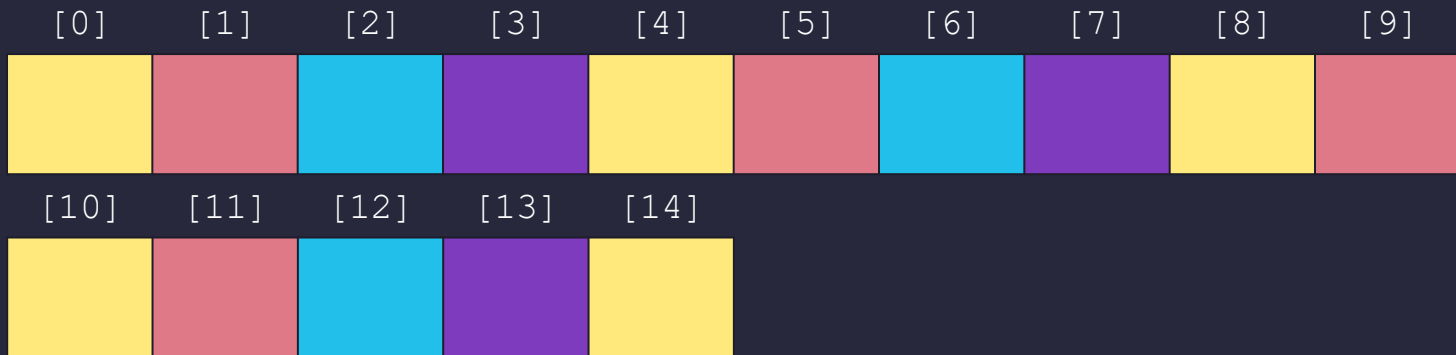
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
22	39	2	73	27	10	19	90	44	13

/INSERINDO ELEMENTOS

```
myList.add(50)
```

Na versão atual do Java, ao inserir um elemento que excede o tamanho atual do array, o programa cria um novo array, aumentando em 50% a capacidade atual.

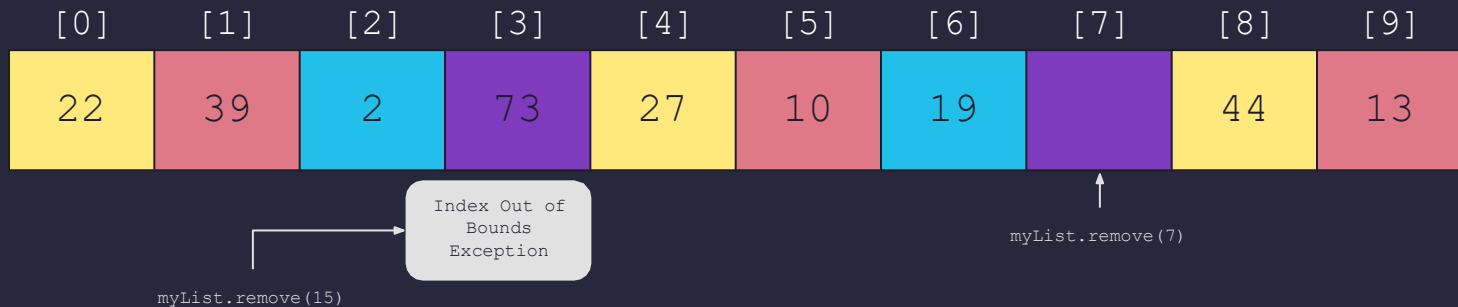
Isso significa que o novo array terá tamanho 15.



/REMOVENDO ELEMENTOS

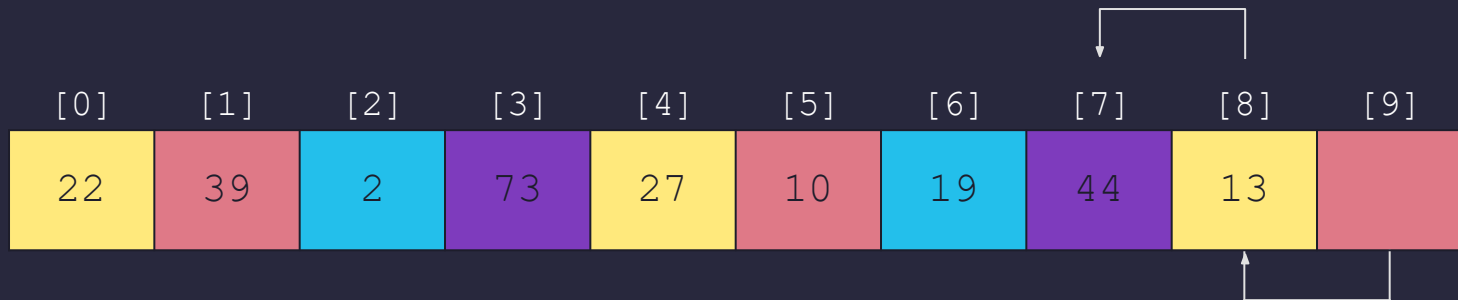
```
myList.remove(idx)
```

Podemos remover um elemento pelo índice. No caso, se for um índice válido, remove o elemento. Caso contrário, dispara um erro (*IndexOutOfBoundsException*).



/REMOVENDO ELEMENTOS

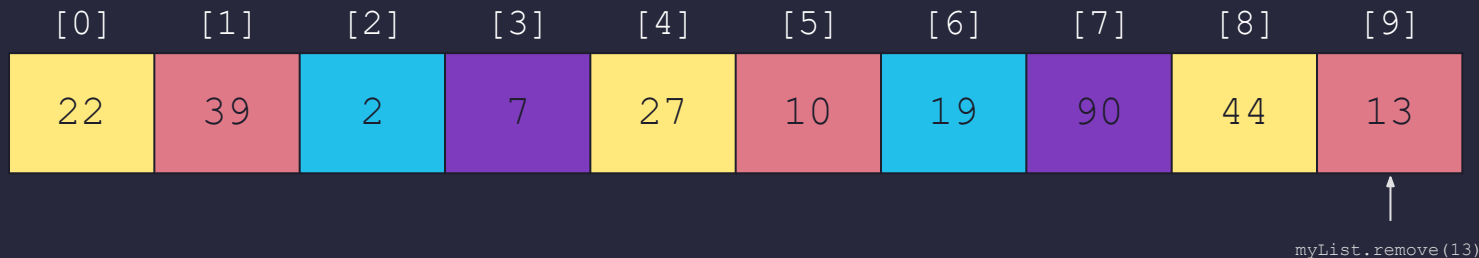
Observem que nesse caso foi deixado um espaço entre os elementos, mas não é isso que acontece na implementação do **ArrayList**. Portanto, é preciso verificar se existem outros elementos posteriores ao elemento removido e realocar esses elementos para que a lista siga de forma contínua, sem espaços entre os elementos inseridos.



/REMOVENDO ELEMENTOS

```
myList.remove(e)
```

Outra forma é remover informando diretamente o elemento. Primeiro fazemos uma busca pelo elemento, caso for encontrado é removido. Caso contrário, apenas não remove e o método retorna *false*.



/“FLUXOGRAMA”

ArrayList

```
Object[] data;  
int capacity;
```

```
ArrayList();  
ArrayList(int capacity);  
add(E e);  
add(int index, E e);  
remove(int index);  
remove(Object o);  
set(int index, E e);  
get(int index);  
contains(Object o);  
indexOf(Object o);  
toArray();
```

set: Sobrescreve o elemento do índice por outro elemento.

get: Retorna o elemento no índice.

contains: Retorna *true* se o elemento for encontrado e *false* caso contrário.

indexOf: Retorna o índice de um elemento.

toArray: Retorna um array estático com os elementos atuais do ArrayList.

/EXERCÍCIOS

1) Explique a diferença entre tamanho e capacidade em um ArrayList.

2) Ao remover um elemento de um ArrayList por índice, o que acontece com os elementos seguintes na lista? Qual a complexidade dessa operação?

3) Explique a diferença entre `remove(int index)` e `remove(Object o)` no ArrayList. Dê um exemplo em que o uso de números pode confundir o programador.

4) Considere o seguinte código: `ArrayList lista = new ArrayList<>(); lista.add(2); lista.add(4); lista.add(6); lista.add(8); lista.add(10); lista.add(2, 99); System.out.println(lista);` Qual será a saída no console? Explique o que aconteceu.

/EXERCÍCIOS

5) Dada a lista: ArrayList
lista = new ArrayList<>();
lista.add(7);
lista.add(9);
lista.add(11);

lista.add(13);
lista.add(15);
lista.remove(1);
System.out.println(lista);
Qual será a saída?
Explique por quê.

6) Escreva um código que insira 12 elementos consecutivos (0 até 11) em um ArrayList criado sem parâmetro de capacidade inicial.

– Qual a capacidade inicial? – Para qual capacidade o ArrayList crescerá ao ultrapassar os 10 elementos?

/EXERCÍCIOS - PESQUISA

7) Explique a diferença entre ArrayList e LinkedList.

8) Quais são as principais vantagens e desvantagens de usar LinkedList?

9) Cite dois métodos comuns da classe LinkedList e explique para que servem.